Taktile claims triage: how the system works

Written for: the team, in plain language. Part B (page after this one) is the technical reference for coding agents.

1. What we are building

A tool that reads one motor claim at a time and recommends one of three next steps: **routine processing**, **ask the claimant for information**, or **send to a human reviewer**. It always shows which rule decided, the facts it used, and the policy version. When the policy changes, it shows which past and open claims would change before the new policy goes live.

2. The three steps a claim goes through

Step	What it checks	Possible outcome
1. Known red flags (Rule 1)	Policy active on the incident date? Motor own-damage claim? Quote at or below EUR 1,500? No injury, liability dispute, duplicate or evidence conflict?	Any flag fails: **human review**
2. Missing information (Rule 2)	Incident report present? Itemised quote present? Any required field blank?	Anything missing: **request information**
3. Everything else (Rule 3)	Nothing above applies	**Routine processing** (this does not approve payment)

Blank fields are treated as missing facts. We never guess a value. A blank review field leads to a request for information, not to routine processing.

3. The four things that make it trustworthy

a) Every decision explains itself. Each one stores the rule that fired, the facts used, the policy version and any blank fields. A reviewer can see why the system decided what it did.

b) Policy changes are tested before release. A proposed policy is run against every stored claim. The system lists every claim whose outcome would change. Open claims are flagged for recheck. Closed claims keep their original decision. Any change that would move a claim into routine processing needs a named sign-off before release.

c) Output is checked before upload. The results file is validated for the exact column names, no duplicate or missing claims, allowed decision values and non-empty explanations, and a checksum is saved.

d) Reviewer decisions are recorded, never silently applied. An override stores who, what, why and which policy version, in a log that can only be added to. Overrides do not change the rules by themselves.

4. Status: what works today

Part	Status	Evidence
Rules engine (3 rules, blanks as missing)	BUILT + TESTED	Agrees with the written policy on all 14 dev cases, 10,000 synthetic and 137 adversarial cases
Change replay with flip list and sign-off gate	BUILT + TESTED	Never changes a closed claim; flips correct on test changes
Output file validator	BUILT + TESTED	Catches 5 kinds of defect in tests
Per-claim trace (facts and rule)	BUILT + TESTED	Decision recomputes exactly from stored facts
Append-only decision log	BUILT + TESTED	Refuses invalid entries; never rewrites earlier ones
Batch command: CSV in, validated CSV out	NOT BUILT	The pieces exist but are not joined into one command. This is the top priority.
Reviewer override screen	NOT BUILT	The log exists; nothing writes to it from a screen yet
Business case sheet	NOT BUILT	Formulas and sensitivity results exist; no sheet yet

5. What we can and cannot claim

We can say: the system follows the written policy consistently; policy changes are tested before release; every decision is explained and recorded.

We cannot say yet: that it is accurate on real claims or on the judges' hidden cases. The large test sets were labelled by the same policy, so agreement shows consistency, not correctness. The 14 real dev cases are the only independent labels, and they are too few to measure accuracy.

Assumptions, not measurements: blank-field rate (3%), quote size distribution (median EUR 800), and every business-case number. Each is labelled as an assumption wherever it appears.

6. Questions for the Taktile mentors

1) Should a blank review trigger go to request-information (our reading) or to review? 2) When the claim summary and a structured field disagree, which one decides? 3) Is a claim whose quote is exactly EUR 1,500 routine (our reading: yes)?

Part B: technical reference (for coding agents)

Working folder: scratchpad taktile-materials. **Status legend:** BUILT + TESTED = test in test_all.py passes; BUILT (not end-to-end) = code exists but no single command runs it; NOT BUILT = do not assume it exists.

B1. Constraints (read before any change)

- No code, text, thresholds or formulas copied from VELTH, veOS, Final-Hack-EHL, BMW-Ride-Planner, Paris-EHL-Hack or any other team repository. Design ideas only, reimplemented from DESIGN.md.
- No organiser or private hackathon data used anywhere. The 14 dev cases are the only organiser labels.
- Nothing is committed, pushed or published without the user explicitly asking.
- Every result states its data source and whether labels are independent or circular.

B2. Modules

File	Role	Status	Test
triage.py	Original v1 engine; parsing helpers (parse_bool, REVIEW_BOOL_FIELDS, EVIDENCE_BOOL_FIELDS, QUOTE_LIMIT_EUR=1500)	BUILT + TESTED	test_all: engine parity
engine.py	decide(row, quote_limit=1500) -> (recommendation, rule_id, explanation). Rules R1 review, R2 request info, R3 routine	BUILT + TESTED	parity with triage.py; oracle agreement
replay.py	replay(claims, new_quote_limit) -> (results, flips, blocked). Closed claims keep v1; flip into routine goes to blocked	BUILT + TESTED	zero-change, closed-never-flipped, flip reasons, blocked
validator.py	validate(results_path, input_ids) -> list of errors (empty = OK); sha256(path)	BUILT + TESTED	5 defect cases
trace.py	write_trace(rows, path); read_trace(path); replay_from_trace(entry)	BUILT + TESTED	round trip
decision_log.py	DecisionLog(path).append(record); .read(). Append-only, validated types and reason	BUILT + TESTED	earlier bytes unchanged; refusals
guard.py	escalate_only(rule_rec, model_flag): never less cautious than the rule. Off by default.	BUILT + TESTED	monotone over all combinations
run.py	Intended: CSV in -> engine -> validator -> results CSV + trace	NOT BUILT	-
reviewer screen	Intended: show claim, rule, facts; override form with reason; writes decision_log	NOT BUILT	-
business_case	Intended: spreadsheet with assumption register	NOT BUILT	-

B3. Data contracts

Input CSV columns (in order):

```
case_id, case_status, claim_summary, policy_active_on_incident, claim_type, repair_type,
quote_eur, incident_report_present, itemized_quote_present, injury_reported, liability_disputed,
duplicate_flag, evidence_conflict. Booleans are the strings True/False; a blank cell means absent.
```

Results CSV columns (exact order, validator enforces it):

```
case_id, recommendation_v1, recommendation_v2, needs_recheck, rule_v1, rule_v2, explanation.
recommendation values: routine_processing | request_information | human_review. needs_recheck:
True | False. Closed claims: recommendation_v2 = retained v1 decision, needs_recheck = False.
```

`Decision log record:` `type (reviewer_override | policy_decision), case_id, actor, reason (non-empty), policy_version, timestamp (added by the log). Stored as one JSON object per line.`

`Trace record:` `case_id, policy_version, decision, rule_id, explanation, facts{...}, blank_fields[...]. One JSON object per line.`

B4. Policy v1 logic (exact order, first match wins)

R1 human_review if ANY of: policy_active_on_incident is False; claim_type is set and not motor_own_damage; injury_reported, liability_disputed, duplicate_flag or evidence_conflict is True; quote_eur > 1500.
R2 request_information if none of R1, and ANY of: incident_report_present or itemized_quote_present is False or blank; quote_eur blank or unreadable; policy_active_on_incident blank; claim_type blank; any review boolean blank.

R3 routine_processing otherwise (does not approve payment).

B5. Replay and gate

replay(claims, new_quote_limit): recomputes v1 with 1500 and v2 with new_quote_limit for open claims. Flips are open claims whose recommendation changes. blocked = flips into routine_processing. A release is blocked until each blocked flip has a named sign-off. Demo limits tested: 1000, 1200, 1300, 1400, 1450, 1500 (no change), 1550, 1600 (1550 and 1600 move claims into routine).

B6. Evidence summary (for claims made in the pitch)

Result	Data	Label source	Status
Engine agrees with written-policy oracle	10,000 synthetic, 137 adversarial	Circular (same policy text)	Consistency, not accuracy
Engine agrees with case-file labels	14 dev cases	Organiser case file (independent, tiny)	14/14; too few to measure accuracy
Sealed hidden-test models	3,000 hidden of 10,000	Circular	Shows learnability only
Blank-escalation cost	Synthetic, sweep over blank rate	Assumed blank rate	Depends on assumption: 0% to 41% unneeded review
Replay flip counts	Synthetic, sweep over limit	Made-up v2 limits	Demonstrates mechanism
Routine audit gate (183 claims, zero errors, two-sided 95%)	None yet	Would need real audits	Design only

B7. Commands

```
cd scratchpad/taktile-materials
python test_all.py # 20 tests, expect ALL TESTS PASSED
python simulate_all.py # candidate table, writes results.json
python sensitivity.py # sweeps on 5,000 claims (fast)
python full_sensitivity.py # full 10,000 file, CIs, holdout (slow)
python hidden_test.py # sealed split, writes hidden_test_ids.txt
```

B8. Next work, in priority order

1. run.py: one command, input CSV to validated results CSV plus trace; run on practice cases; validator must pass.
2. Reviewer screen: claim view, override form with required reason, writes decision_log.
3. Business case sheet: assumption register (each value tagged assumed/measured), escalation formula, cost of a policy change using the flip list.
4. Pitch and recorded fallback demo. Run the truth gate on every claim before it goes in.